Vehicle Maintenance and Repair Management System

Problem Statement

In many vehicle service workshops or personal maintenance routines, vehicle maintenance records are still managed manually using paper or basic spreadsheets. This method is inefficient, error-prone, and makes it difficult to track service history, upcoming maintenance, or cost management over time.

There is a clear need for a **centralized digital solution** that allows users to:

- Record and access vehicle service and repair details quickly
- Maintain a history of repairs, parts replaced, and associated costs
- Set reminders for periodic maintenance
- Help workshop administrators manage vehicles, customers, and staff efficiently

The **Vehicle Maintenance and Repair Management System** aims to solve this problem by providing a user-friendly, web-based solution that ensures organized, searchable, and secure data storage related to vehicle servicing and repairs.

Project Scope: Vehicle Maintenance and Repair Management System

Project Overview

The **Vehicle Maintenance and Repair Management System** is a web-based application designed to help users efficiently manage the maintenance, repair schedules, and service history of vehicles. The system aims to digitize and streamline manual vehicle service record-keeping through a user-friendly interface.

This system will be built using:

- **Frontend:** HTML, CSS, JavaScript
- **Backend:** PHP
- **Database:** MySQL

Objectives

- To allow users to manage inventory

- To track vehicle status.
- To manage repair jobs, parts used, and customer details.
- Allow to store customer data more accurately

Key Features

1. **Vehicle Registration** – Add and manage vehicle details like model, number plate, owner, etc.
2. **Maintenance Logs** – Record each service or repair with the date, description, cost, and parts used.
3. **Admin Dashboard** – Secure backend interface to manage vehicles, repairs, users, and database operations.
4. **Customer data** – Easily store data of all customers .

Technology Scope

- **Frontend**:
  - **HTML** for structure
  - **CSS** for styling and layout
  - **JavaScript** for form validation and interactivity
- **Backend**:
  - **PHP** for handling form submissions, backend logic, and database interaction
- **Database**:
  - **MySQL** to store all vehicle, service, mechanic, and parts information

Out of Scope

- Integration with GPS tracking or telematics
- Mobile app development

Target Users

- Workshop administrators and mechanics
- Vehicle maintenance staff in institutions

Project Setup and Installation Guide

Prerequisites

- **XAMPP or WAMP** installed on your system

- MySQL Workbench installed
- Project files ready to be deployed
- .sql database file included in the project
- A browser (e.g., Google Chrome)

Step 1: Install XAMPP

1. Download the latest version of **XAMPP** from the official website: https://www.apachefriends.org/index.html
2. Run the installer and follow the on-screen instructions.
3. Once installed, open the **XAMPP Control Panel**.

Step 2: Open SQL File in MySQL Workbench

1. Launch **MySQL Workbench**.
2. From the home screen, click on the **"+"** icon next to *MySQL Connections* to create a new connection.
3. Provide a connection name and use the following default credentials:
    - Hostname: localhost
    - Port: 3306
    - Username: root
    - Password: *(leave blank unless configured otherwise)*
4. Click **Test Connection** to ensure connectivity.
5. Click **OK** to save the connection.
6. Open the .sql file using the **File > Open SQL Script** option or drag and drop the file into the query editor.
7. Select your connection and click **Run** to execute the script and create the database.

Step 3: Configure db.php File

1. Open the project folder and locate the file named db.php.
2. Ensure the following MySQL credentials are correctly configured:
3. Make sure that you enter your details correctly in the bd.php file.

db.php

CopyEdit

$host = *localhost*;

```
$user = root;

$password = 'Your_database_passowrd'

$database = 'your_database_name';

$conn = mysqli_connect($host, $user, $password, $database);
```

3. Save the changes.

Step 4: Move Project Files to XAMPP Directory

1. Copy the entire project folder.
2. Navigate to the directory:
   C:\xampp\htdocs\
3. Paste the project folder inside the **htdocs** directory.

Step 5: Start Apache and MySQL Services

1. Open the **XAMPP Control Panel**.
2. Click **Start** next to both **Apache** and **MySQL** modules.
3. Ensure both services show **green indicators**, confirming they are running.

Step 6: Verify Database in phpMyAdmin

1. Open a web browser and visit:
   https://localhost/phpmyadmin
2. Confirm that your database (imported in Step 2) appears in the left sidebar.

Step 7: Run the Project in Browser

1. Open a new browser tab.
2. Type the following URL in the address bar:
   https://localhost/YOUR_PROJECT_NAME/
3. The homepage of your project should load, confirming successful installation.

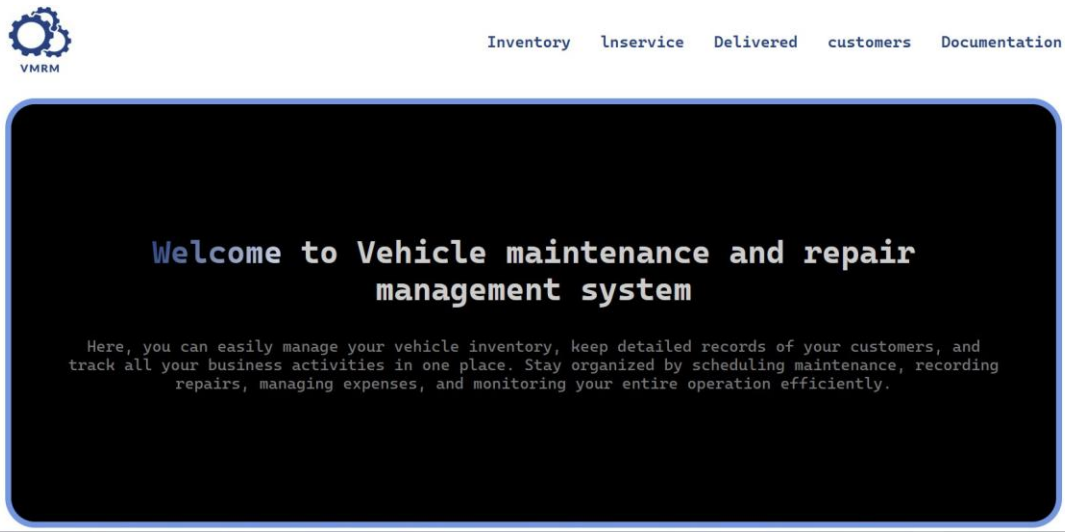
SCREENSHOTS AND SOURCE CODE

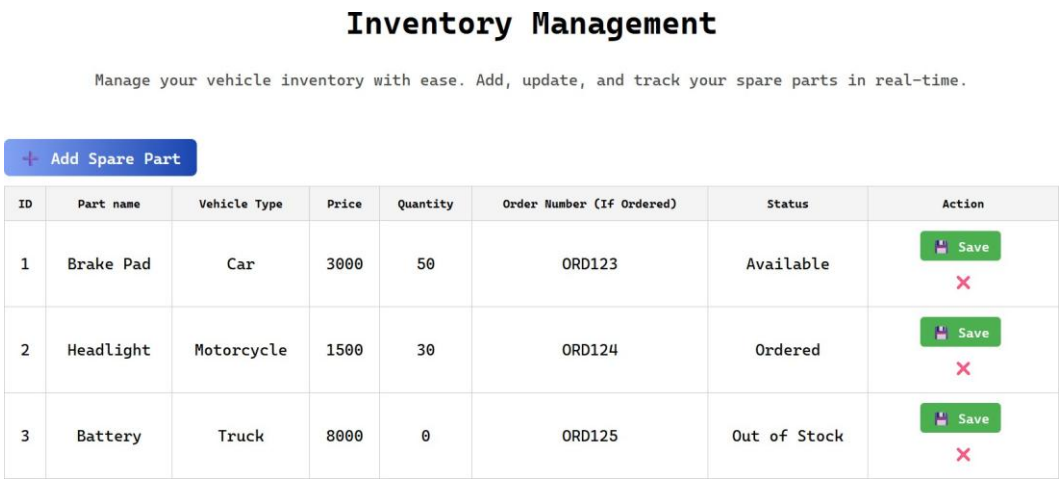
Github link:

https://github.com/Wajeeh001/VMRM

You can download the project code by clicking on this link.

Screenshots:





Vehicle status / Repaired

Delivered Vehicle / Repaired ensures your vehicle is fully repaired and delivered back to you in optimal condition.

+ Add Vehicle status

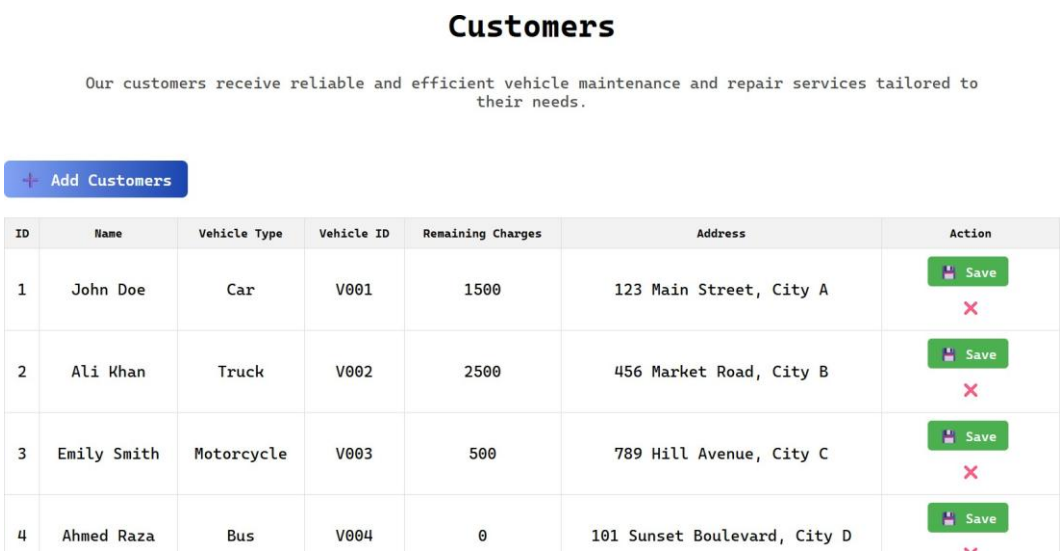
ID	Vehicle ID	Type Vehicle	Changed Parts	Total Cost	(Paid/Not)	Delivery Status	Action
1	V001	Car	Brake Pad	100	Paid	Delivered	Save ✕
2	V002	Truck	Battery	200	Not Paid	Not Delivered	Save ✕
3	V003	Motorcycle	Headlight	300	Paid	Delivered	Save ✕

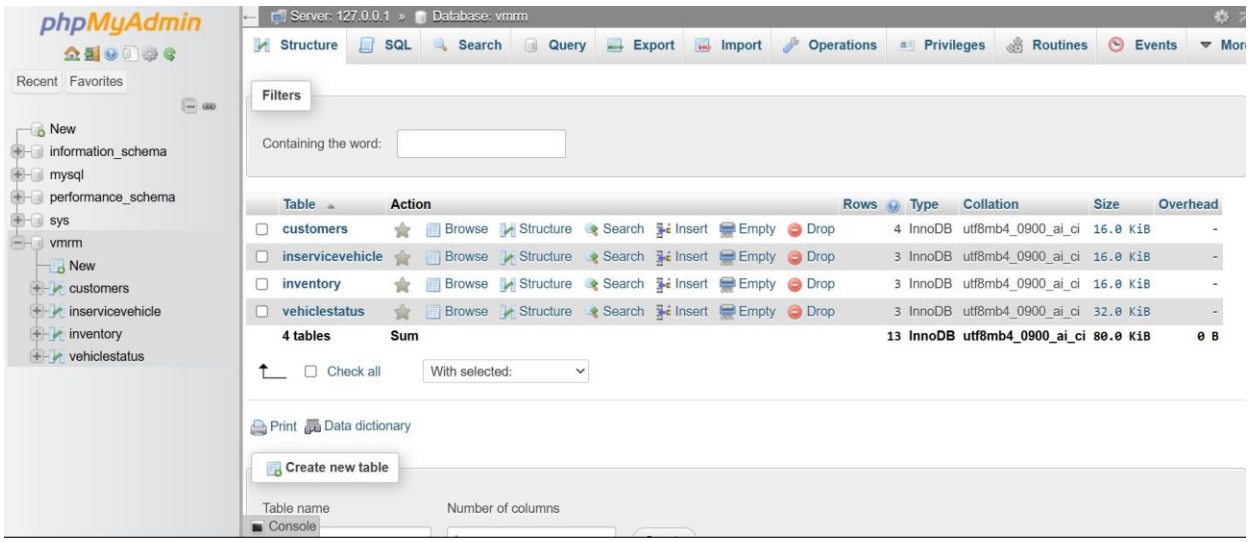
Inservice Vehicle

Inservice Vehicle offers top-notch maintenance and repair management to keep your vehicles in peak condition

+ Add Inservice

ID	Vehicle ID	Type Vehicle	Changed Parts	Labour Charges	Parts Charged Cost	Total Cost	Delivery Date	Action
1	V001	Car	Brake Pad	500	3000	3500	2025-05-01	Save ✕
2	V002	Truck	Battery	1000	8000	9000	2025-05-02	Save ✕
3	V003	Motorcycle	Headlight	300	1500	1800	2025-05-03	Save ✕





Understanding How Data is Handled in Our System

1. Where Does the Data Go?

Whenever someone fills out a form on our website and submits it, that information doesn't just vanish — it's sent straight to **phpMyAdmin**, which is where all the data is stored and managed. Behind the scenes, PHP takes care of passing that data into our database.

2. What Happens in the Backend?

The backend (which is written in PHP) has instructions on what to do when someone performs an action like clicking "Save" or "Delete." Based on what the user does, PHP will create a

specific query (like "Insert this info" or "Delete this record") and send it to MySQL to perform the action.

3. What Does MySQL Do?

MySQL is the database engine. Once it gets the query from the backend, it does exactly what it’s told — whether that’s saving new data, deleting old data, or updating something. If everything goes well, it lets the system know the job is done; if not, it returns an error.

4. Saving vs. Deleting — Separate Actions

To keep things clean and organized, we’ve written separate code for **saving** and **deleting** data. That means:

- There's one section of the code (or one function) just for **saving** new entries.
- And a different section for **deleting** data.

This makes the system easier to manage and troubleshoot later.